Agent Evaluation with MLFLow and EvalHub on Red Hat OpenShift AI

Version 1, 2026-07-29

Home

Goal

This course teaches how to deploy MLflow and EvalHub as included with Red Hat OpenShift AI 3.4; how to run evaluations of a self-hosted model, using EvalHub; how to run evaluations of an agentic application, using the MLflow SDK with Python; and how to visualize the results of those evaluations as MLflow experiments and traces using the Red Hat OpenShift AI dashboard.

Total estimated reading time: ...

You should reserve between 4 hours to one working day to complete this course.

Red Hat associates must enroll using the Red Hat Learning Portal, and Red Hat Partners must enroll using Red Hat Partner Connect to register this course in their learning history.

Objectives

On completing this course, you should be able to:

- Understand how MLflow and EvalHub relate to agentic applications and to Red Hat OpenShift AI
- Deploy a development environment with MLflow and EvalHub
- Evaluate LLMs using EvalHub
- Evaluate agents using MLflow

Audience

- Software developers working on agentic applications and agentic harnesses.
- Red Hat and Partner roles that perform demonstrations, PoC and PoV of MLflow with Red Hat OpenShift AI, such as Solutions Architects.
- Red Hat and Partner roles that implement and support MLflow and EvalHub with Red Hat OpenShift AI, such as Services Consultants, Technical Account Managers, and Customer Support Engineers.

Prerequisites

This course assumes that you have the following experience:

- Familiarity with generative AI and agentic application concepts.
- Familiarity with the basic operation of Red Hat OpenShift AI.
- Python programming skills are recommended but not required, because this training provides sample code which is ready to run.

It is recommended, but not required, to attend a prerequisite course about agentic traces with MLflow.

Classroom environment

This course uses the Red Hat OpenShift AI 3.4 demo catalog entry, which provides an SNO OpenShift cluster based on an AWS instance with a single GPU and a predeployed llama-32-3b-instruct model.

Any additional components required by MLflow, EvalHub, or course activities are configured by learners as part of course activities, using sample Python code and Kubernetes manifests provided in a public Git repository.

Other sources of information about MLflow and EvalHub

For documentation about Red Hat OpenShift AI, see its product documentation, especially:

- Working with MLflow
- Chapter 2 of Evaluating AI systems: Evaluate LLMs with EvalHub

Also refer to upstream documentation ofhttps://mlflow.org/docs/latest/index.html[MLflow^] and EvalHub.

Author

Fernando Lozano
Training Content Architect
Red Hat - Product Portfolio Marketing & Learning

Lab Environment

FIXME: Instructions for accessing the lab environment for this hands-on based training.

Introduction to MLFlow and EvalHub

Goal

Understand how MLFlow and EvalHub relate to agentic applications and to Red Hat OpenShift AI.

Not started yet.

Evaluation Concepts for AI Applications

Objective

Describe how evaluating AI-enabled applications differs from testing traditional applications and identify the roles of MLFlow and EvalHub in Red Hat OpenShift AI.

Not started yet.

Heading

Lorem ipsum

Another heading

Lorem ipsum

What's Next

Now that you understand the key evaluation concepts for AI applications, test your knowledge with a quiz on MLFlow and EvalHub fundamentals.

Lab: Explore the Learning Environment

Objective

Navigate the provided demo environment, identify enabled features of Red Hat OpenShift AI 3.4, and verify the predeployed llama model.

Pending Review

Before you begin

You need access to the predeployed Red Hat OpenShift AI (RHOAI) instance provided in your lab environment. Ensure you have:

- Access credentials for the RHOAI dashboard
- A web browser to access the RHOAI dashboard
- A web browser to access the OpenShift web console

Instructions

In this lab, you will log in to the RHOAI dashboard and explore its main features and components to understand the platform's capabilities. You will also install the Web Terminal operator in your OpenShift cluster to use for activities which require command-line tools.

Inspect the existing projects and deployed models

1. Access the RHOAI Dashboard.
   a. Navigate to the RHOAI dashboard URL of your demo environment.
   b. Log in using the `admin` user, from the credentials provided for your demo environment.
2. Explore the precreated project.

   Your RHOAI instance contains a model which is already deployed in a regular project named `my-first-model` — which is **not** an AI project.

   That model takes up all GPU capacity available to your RHOAI instance, so any change to the model deployment requires that you stop it and restart after making changes.

   a. In the left pane, select **AI hub › Models** and then select the **Deployments** tab.
   b. In the **Project** field, enter `my` and select the **my-first-model** project.

   

   Depending on the current page of the RHOAI dashboard, it may be necessary to first select `All projects` before you can search and select the `my-first-model` project.

   c. Verify that there is a model deployment named `llama-32-3b-instruct` and its status is `Ready`.
   d. Click **View**, under the **Endpoints** column for the model row, and verify that it provides only an internal endpoint. This means the model cannot be accessed from outside of its OpenShift cluster, unless you change its model deployment.
3. Try the model with a Gen AI studio playground.
   a. In the left pane, select **Gen AI studio › Playground**.
   b. In the **Project** field, enter `my` and select the **my-first-model** project.

   

   The project previously selected in one page of the RHOAI dashboard may not be the current selection in another page of the RHOAI dashboard.

   c. If you do not see a **Configure** pane, click **Settings**. In the **Configure** pane, select the **Model** tab and verify that the selected model is named `llama-32-3b-instruct`. Ignore the prefix on the model name.
   d. In the right pane, click **Send a message...** and type a prompt, for example:

   ```
   What is RHEL?
   ```

e. Type `Enter` to submit the prompt and see the response from the model. The model may return different answers if you attempt the same prompt again, but they all should be correct and reasonable.

Confirm that MLflow and EvalHub are not deployed in RHOAI

4. Verify that there are no RHOAI dashboard pages related to MLflow.
   a. In the toolbar, to the top right, click the **Applications** icon, which is next to the **Question** icon, and verify that there is **not** an entry named **MLflow**.
   b. In the left pane, expand the **Develop & train** category, which contains entries named **Feature store**, **Pipelines**, **Jobs**, and **Evaluations**.
   c. In the left pane, expand the **Gen AI studio** category, which contains entries named **Playground** and **AI asset endpoints**.

      Enabling MLflow will add additional entries here.

5. Verify that there are no MCP servers configured in the GenAI Studio.

   Select **Gen AI studio › AI asset endpoints** and then select the **MCP servers** tab.

   The page should list no MCP servers.

6. Verify that the Evaluations page from RHOAI dashboard is not functional.
   a. In the left pane select the **Develop & train › Evaluations**.
   b. The **Evaluations** page display the messages `Evaluations unavailable` and `To use evaluations, enable the evaluation service using the Trusty AI operator`.

Install the Web Terminal operator

7. To avoid issues with local installation of Python in your regular work computer, install the Web Terminal operator on your OpenShift cluster.
   a. Navigate to the OpenShift web console URL of your demo environment. It should be automatically logged in as the `admin` user that you already authenticated to the RHOAI bashvoard.
   b. In the left pane, select **Ecosystem › Software Catalog**. Enter `web terminal` in the search field.
   c. Click **Web Terminal** provided by Red Hat and, on the operator details page, click **Install**.
   d. In the **Install Operator** page, make no changes and click **Install**. When it displays the `Manual approval required` message, click **Approve**.
   e. When you see the message `Operator installed successfully`, refresh your web browser tab to reload the HTML and JavaScript content of the OpenShift web console.
8. Verify that the web terminal provides the OpenShift CLI.
   a. Verify that the toolbar displays the <menu>>_</menu> icon, with tooltip <code>OpenShift command line</code> next to the <menu>help</menu> or <menu>?</menu> icon.
   b. Click the >_ icon, which opens the **Terminal 1** tab at the bottom of your OpenShift web

console and click **Start**.

c. When the terminal displays a bash prompt, enter the following commands to verify that you have, from the terminal, cluster administrator access to your OpenShift cluster.

GRAB OUTPUT

```
$ oc version
Client Version: 4.20.0-202605211651.p2.g02b0b2d.assembly.stream-02b0b2d
Kustomize Version: v5.6.0
Server Version: 4.20.29
Kubernetes Version: v1.33.13
$ oc whoami
admin
$ oc get node
NAME                          STATUS   ROLES                         AGE
VERSION
ip-10-0-30-252.ec2.internal   Ready    control-plane,master,worker   7h8m
v1.33.13
```

Summary

After completing these steps, you verified that you have a functional RHOAI instance with a pre-deployed model, which is available only for internal access, and that you can submit prompts to the model using an RHOAI playground.

In addition, you have also verified that MLflow and EvalHub are is not installed on your RHOAI instance, and that there are no MCP servers deployed.

What's next

Now that you are familiar with the learning environment, the next chapter covers how to deploy MLFlow and EvalHub for development and testing usage on Red Hat OpenShift AI.

Deploy MLFlow and EvalHub

Goal

Deploy a development environment with MLFlow and EvalHub on Red Hat OpenShift AI.

Not started yet.

MLFlow Architecture and Integration

Objective

Describe the architecture of MLFlow and explain how to enable and access it in Red Hat OpenShift AI.

Not started yet.

Heading

Lorem ipsum

Another heading

Lorem ipsum

What's Next

Next, you deploy and configure MLFlow on your Red Hat OpenShift AI instance.

Lab: Deploy MLflow for Development

Objective

Deploy and verify a development instance of MLflow in Red Hat OpenShift AI.

Work In Progress

Before you begin

You need access to the predeployed Red Hat OpenShift AI (RHOAI) instance provided in your lab environment. Ensure you have:

- Access credentials for the RHOAI dashboard
- A web browser to access the RHOAI dashboard
- Access credentials for the OpenShift web console

If you are in a hurry and need to quickly enable MLflow in your demo instance, for example: you had to destroy and recreate it before completing all activities in

this course, you can:

- Clone the samples git repository
- Enter its `scripts` directory
- Log in to OpenShift in your demo instance as the `admin` user, using the `oc` command
- Invoke the `enable-mlflow.sh` script

Instructions

You will enable MLflow on your Data Science Cluster singleton resource and deploy a development instance of the MLflow server. Then you will create a Jupyter notebook to test connectivity to the MLflow server. You will use the MLflow SDK for Python.

As you perform this activity, you will alternate between three browser tabs:

- The Red Hat OpenShift web console
- The Red Hat OpenShift AI (RHOAI) dashboard
- A Jupyter notebook from a RHOAI workbench

Once you open one of these tabs, keep it open because you may return to it later.

Install and configure the MLflow Server

1. Enable MLflow on RHOAI by editing its data science cluster resource.
   a. Open the OpenShift web console and log in as a user with `cluster-admin` rights.
   b. In the left navigation pane, select **Ecosystem › Installed Operators**. Then click **Red Hat OpenShift AI**.
   c. In the Red Hat OpenShift AI operator page, select the **Data Science Cluster** tab. Then click the **default-dsc** resource.
   d. In the `default-dsc` resource page, select the **YAML** tab. In the text editor, search for the `spec.components` field, and set `mlflowoperator.managementState` to `Managed`.

```
...
spec:
  components:
    mlflowoperator:
      managementState: Managed
    kserve:
      managementState: Managed
      rawDeploymentServiceConfig: Headless
...
```

   e. Click **Save** and wait until you see a confirmation message similar to:

```
alert:default-dsc has been updated to version 1186240
```

f. If you receive an error message, fix the errors in your YAML and try saving again.

2. Configure your MLflow instance by creating a MLflow resource.

a. Open the sample MLflow manifest from the course samples repository. Copy the entire contents of the `scripts/mlflow.yaml` manifest to the clipboard.

That manifest corresponds to the example from Minimal development or test deployment in the Red Hat OpenShift AI product documentation with no changes.

The full manifest is listed here, just for your reference:

```
apiVersion: mlflow.opendatahub.io/v1
kind: MLflow
metadata:
  name: mlflow
  namespace: redhat-ods-applications
spec:
  storage:
    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: 10Gi
  backendStoreUri: "sqlite:////mlflow/mlflow.db"
  artifactsDestination: "file:///mlflow/artifacts"
  serveArtifacts: true
```

b. In the left navigation pane, select **Home › API Explorer**. Then enter `MLflow` in the search field. You will see three resource types; click **MLflow**.

c. In the MLflow Resource details page, select the **Instances** tab and click **Create MLflow**.

d. In the YAML text editor, delete all of its sample contents and paste the manifest you already copied to the clipboard.

e. Click **Create** and, in the MLflow details page, wait until the `Available` condition is `True` and the `Progressing` condition is `False`.

Test your MLflow server using the dashboard and a workbench

3. Verify MLflow is available from RHOAI and create an empty experiment.

a. Open your RHOAI dashboard. If it is already open, refresh your browser window.

b. In the left navigation pane, expand **Develop & train**. Notice the new item named **Experiments (mlflow)** and click it.

c. In the **Project** field, select the **my-first-model** project.

d. Click **Create Experiment** and enter `dummy experiment` as its name.

e. Verify that you see the Experiments page for `dummy experiment` but there is no data on any of its pages, such as **Traces**, **Sessions**, or **Prompts**.

f. Creating an empty experiment is sufficient to prove that your MLflow instance can connect to its database and artifacts storage.

g. You can, alternatively, open the upstream MLflow web interface by clicking the **applications** menu, near the top right of the navigation bar close to the help icon, and selecting **MLflow**. You should not need it: all functionality you need for this course is available from the RHOAI dashboard.

4. Test connectivity to MLflow from a Jupyter notebook inside a workbench.

If you have trouble typing the Python code in this and other activities of this course, or you cannot cut-and-paste from the course page, you can access the complete source code for all Jupyter notebooks in the course samples git repository.

a. In the left navigation pane, click **Projects** and select **All projects** in the **Project** field. Then enter `my` in the **Filter by name** field and click **my-first-model** in the list of projects.

b. In the Projects page for `my-first-model`, select the **Workbenches** tab and click **Create workbench**.

c. In the Create workbench page, enter `test-evals` for the **Name** field and, in the **Workbench image** field, select **Jupyter** | **Minimal** | **CPU**.

d. Leave all other fields with their default values and click **Create workbench**.

e. In the Workbenches list, wait until the row for the `test-mlflow` workbench displays a `Ready` state and then click **test-mlflow** to open its Jupyter Lab in a new browser tab.

f. Click the **Python 3.12** Notebook icon to create an empty notebook. In the left navigation pane of the Jupyter Lab, right-click the **Untitled.ipynb** file, which is your new notebook, and select **Rename**. Then, enter `test-mlflow.ipynb` as its new file name.

g. In the first cell of your notebook, enter:

```
! pip install mlflow[kubernetes]
```

h. In the top toolbar from the notebook, click the **play** icon, having the tooltip "Run this cell and advance".

i. Wait until the MLflow SDK and all its dependencies are loaded from the RHOAI index into the workbench.

j. After the `pip` command finishes, enter the following on the second cell of your notebook:

```
import os
import mlflow

os.environ["MLFLOW_TRACKING_AUTH"]="kubernetes-namespaced"
```

```
mlflow.set_tracking_uri("https://mlflow.redhat-ods-
applications.svc.cluster.local:8443")

print(mlflow.list_workspaces())
```

In this example, we are using the *internal* URL of the MLflow server, which works only for applications running in the same OpenShift cluster.

k. In the top toolbar from the notebook, click the **play** icon. The results should be an array containing a single `Workspace` object named `my-first-model`.

   This notebook takes advantage of the integration between the RHOAI dashboard and its managed instance of the MLflow server. It connects using the service account of the `test-evals` workbench, which is authorized to access the MLflow workspace corresponding to its `my-first-model` project.

l. You will not use the `test-mlflow.ipynb` notebook anymore, but do NOT delete the `test-evals` workbench because you will keep using it in the remaining activities of this course.

Summary

After completing these steps, you have verified that you have a minimal but functional MLflow instance, to which you can connect from applications running inside your RHOAI cluster and store data from your experiments.

What's next

With MLFlow deployed and verified, the next section introduces the architecture and setup of EvalHub.

EvalHub Architecture and CLI

Objective

Describe the architecture of EvalHub and explain how to enable, access, and use its CLI with Red Hat OpenShift AI.

Not started yet.

Heading

Lorem ipsum

Another heading

Lorem ipsum

What's Next

Next, you deploy and configure EvalHub on your Red Hat OpenShift AI instance.

Lab: Deploy EvalHub for Development

Objective

Enable EvalHub on Red Hat OpenShift AI using the TrustyAI operator, configure its database, and install and configure the EvalHub CLI.

Work In Progress

Before you begin

You need access to the predeployed Red Hat OpenShift AI (RHOAI) instance provided in your lab environment. Ensure you have:

- Access credentials for the RHOAI dashboard
- A web browser to access the RHOAI dashboard
- Access credentials for the OpenShift web console

Also ensure that you already deployed MLflow by following the instructions from the previous lab because, if you enable EvalHub without having MLflow enabled, then EvalHub will not be able to save metrics on experiments from MLflow.

If you already deployed MLflow, and you are in a hurry and need to quickly enable EvalHub in your demo instance, for example: you had to destroy and recreate it before completing all activities in this course, you can:

- Clone the samples git repository
- Enter its `scripts` directory
- Log in to OpenShift in your demo instance as the `admin` user, using the `oc` command
- Invoke the `enable-evalhub.sh` script

Instructions

You will enable TrustyAI on your Data Science Cluster singleton resource and deploy a development instance of the EvalHub server. Then you will use a shell to test connectivity and healthy of your EvalHub server.

As you perform this activity, you will alternate between three browser tabs:

- The Red Hat OpenShift web console
- The Red Hat OpenShift AI (RHOAI) dashboard
- A Jupyter notebook from the `test-evals` workbench

Once you open one of these tabs, keep it open because you may return to it later.

Install and configure the EvalHub Server

1. Verify that TrustyAI is already enabled in the data science cluster resource.
   a. Open the OpenShift web console and log in as a user with `cluster-admin` rights.
   b. In the left navigation pane, select **Ecosystem › Installed Operators**. Then click **Red Hat OpenShift AI**.
   c. In the Red Hat OpenShift AI operator page, select the **Data Science Cluster** tab. Then click the **default-dsc** resource.
   d. In the `default-dsc` resource page, select the **YAML** tab. In the text editor, search for the `spec.components` field, and verify that `trustyai.managementState` is already set to `Managed`.

```
...
spec:
  components:
    kserve:
      managementState: Managed
      rawDeploymentServiceConfig: Headless
...
    trustyai:
      managementState: Managed
      mcpGuardrailsMode: false
    aipipelines:
      managementState: Managed
...
```

   e. Click **Cancel** to leave the YAML editor without making changes.
2. Deploy a PosgreSQL database.
   a. Open the sample evalhub db deployment from the course samples repository with a web browser and open the manifest `scripts/evalhubdb.yaml`. Copy the entire contents of the manifest to the clipboard.

> This manifest provides a minimal PostgreSQL deployment. It is not listed here because if its size, but you would not use such a simplisitic database deployment for a real development or production instance of EvalHub.
>
> You are free to use your preferred OpenShift template, helm chart, or database operator instead. Ideally, use something that is supported by your organization IT department, and for which you can expect product support from a vendor.

   b. In the left pane, select **Home › Projects** and, in the search field, enter `redhat-ods`. Then click the **redhat-ods-applications** project.
   c. In the toolbar, click the + icon, with tooltip `Quick create` and select **Import YAML**.

d. In the Import YAML page, enter the manifests for a PVC, a deployment, and a service, which you already copied to your clipboard, and click menu:[Create].

e. You should see a page with the message `Resources successfully created`. Click on the **evalhubdb** deployment, the one with a "D" icon.

f. In its deployment details page, wait until you see a blue cicle with a label of "1 Pod", indicating one pod from that deployment is running. Then scrool down to the **Conditions** table, and verify there is a row of type `Available` with status `True`.

3. Configure your EvalHub instance by creating a secret and an EvalHub resource.

a. Open the sample evalhub db secret from the course samples repository with a web browser and open the manifest `scripts/evalhub-db-credentials.yaml`. Copy the entire contents of the manifest to the clipboard.

The full secret manifest is listed here, just for your reference:

```
apiVersion: v1
kind: Secret
metadata:
  name: evalhub-db-credentials
type: Opaque
stringData:
  db-url: "postgres://rhoai:dontdothisinprod@evalhubdb.redhat-ods-
applications.svc.cluster.local:5432/evalhub"
```

b. In the left navigation pane, select **Workloads › Secrets** and verify that the selected project is still `redhat-ods-applications`. Then click **Create › From YAML**.

c. In the **Create Secret** page, enter the manifests for a secret which you already copied to your clipboard, and click **Create**.

d. In the **Secret details** page, you can scroll down and click **Reveal values** to see the database connection URL. It contains the database user name, password, and other connection parameters which match the database deployment you created in the previous step.

e. In the left navigation pane, select **Home › API Explorer**. Then enter `EvalHb` in the search field. You will see only one resource, so click **EvalHub**.

f. Open the sample evalhub manifest from the course samples repository with a web browser and open the manifest `scripts/evalhub.yaml`. Copy the entire contents of the manifest to the clipboard.

That manifest corresponds to the example from 2.3. Deploy EvalHub with the TrustyAI Operator in the Red Hat OpenShift AI product documentation with a couple changes:

- Addition of `lighteval` to the list of providers
- The correct internal URL of the MLflow server

The full manifest is listed here, just for your reference:

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: EvalHub
metadata:
  name: evalhub
spec:
  replicas: 1
  database:
    type: postgresql
    secret: evalhub-db-credentials
  providers:
    - lm-evaluation-harness
    - garak
    - guidellm
    - lighteval
  collections:
    - safety-and-fairness-v1
  env:
    - name: MLFLOW_TRACKING_URI
      value: "https://mlflow.redhat-ods-applications.svc.cluster.local:8443"
```

g. In the EvalHub resource details page, select the **Instances** tab and verify that the selected project is still `redhat-ods-applications`. Then click **Create EvalHub**.

h. In the YAML text editor, delete all of its sample contents and paste the evalhub manifest which you already copied to the clipboard.

i. Click **Create** and, in the EvalHub details page, wait until the `Ready` condition is `True`.

Test your EvalHub server using the RHOAI dashboard

3. Verify that EvalHub is available from the RHOAI dashboard.

   a. Open your RHOAI dashboard. If it is already open, refresh your browser window so it picks changes to the dashoard pages and navigation.

   b. In the left navigation pane, select **Develop & train › Evaluations**. The **Evaluations** page now displays the message `No existing evaluation runs`.

   c. Click **Create Evaluation** and, on the **New evaluation run** page, click **Single benchmark**.

   d. The **Single benchmark** page shows a long, paginated list of benchmarks offered by the providers configured on your EvalHub resource. Do *not* click any of them: Just seeing the list is sufficient to prove that your EvalHub server is functional.

Test your EvalHub server using a shell

4. Verify that your EvalHub server is healthy.

The shell commands in this course assume a Bash shell on Fedora Linux. If your work machine runs Windows or macOS, you should either install a Bash shell or connect to the bastion host of your demo environment using SSH.

a. Open a terminal and log in to the OpenShift cluster from your demo environment, as a cluster administrator, using the `oc login` command.

b. Run the following commands to get the EvalHub URL and access its health endpoint. Your output should include the JSON field `status: healthy`.

```
$ oc project redhat-ods-applications
Now using project "redhat-ods-applications" on server "https://172.30.0.1:443".
$ EVALHUB_URL=https://$(oc get routes evalhub -o jsonpath='{.spec.host}')
$ curl -sk $EVALHUB_URL/api/v1/health
{"status":"healthy","timestamp":"2026-07-
27T23:24:06.894598885Z","build":"0.3.0"}
```

5. Install the EvalHub CLI in a Python virtual environment.

a. Install the EvalHub CLI using `pip`. It should work with any version of Python from 3.12 on.

```
$ python --version
Python 3.14.6
$ python -m venv ~/venv/evalhub
$ source ~/venv/evalhub/bin/activate
... prompt changes ...
$ pip install "eval-hub-sdk[cli]"
...
Successfully installed annotated-types-0.8.0 anyio-4.14.2 certifi-2026.7.22
click-8.4.2 eval-hub-sdk-0.4.4 h11-0.16.0 httpcore-1.0.9
...
```

b. Configure the EvalHub CLI to access your RHOAI instance.

Notice these commands refer to the `EVALHUB_URL` shell variable you set during the previoius step.

```
$ evalhub config set base_url $EVALHUB_URL
Set 'base_url' in profile 'default'
$ evalhub config set tenant my-first-model
Set 'tenant' in profile 'default'
$ TOKEN=$(oc whoami -t)
$ evalhub config set token $TOKEN
Set 'token' in profile 'default'
```

c. Verify that the EvalHub CLI can connect to the EvalHub server and list its known evaluation providers.

```
$ evalhub health
EvalHub service: healthy (640ms)
$ evalhub providers list
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
```

```
┏━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━┓
┃ ID                   ┃ NAME                   ┃ DESCRIPTION
┃ BENCHMARKS ┃
┡━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━┩
│ lm_evaluation_harness │ LM Evaluation Harness │ Comprehensive evaluation
framework for language models with 180 benchmarks │ 181        │
│                       │                        │
│            │
│ lighteval             │ Lighteval              │ Lightweight LLM evaluation
framework from Hugging Face                       │ 25         │
│ guidellm              │ GuideLLM               │ Performance benchmarking
framework for LLM inference servers               │ 7          │
│ garak                 │ Garak                  │ LLM vulnerability scanner
and red-teaming framework                         │ 8          │
└───────────────────────┴────────────────────────┴──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┴────────────┘
```

Summary

After completing these steps, you have verified that you have a minimal but functional EvalHub instance, which you can use to run evaluation jobs.

What’s next

With both MLFlow and EvalHub deployed, the next chapter introduces model evaluation using EvalHub providers and benchmarks.

Model Evaluation with EvalHub

Goal

Evaluate LLMs using EvalHub providers, benchmarks, and collections on Red Hat OpenShift AI.

Not started yet.

Model Evaluation Concepts

Objective

Describe the evaluation providers and benchmarks available in EvalHub and explain how to manage evaluation jobs and interpret their results.

Not started yet.

Heading

Lorem ipsum

Another heading

Lorem ipsum

What's Next

Next, you run evaluation jobs on a self-hosted model and inspect the results.

Lab: Evaluate a Self-Hosted Model

Objective

Run evaluation jobs on a self-hosted model using EvalHub, monitor their progress, and inspect results and metrics from the RHOAI dashboard.

Not started yet.

Before you begin

Describe requirements, such as customer portal accounts, Red Hat Developer free sub, already configured machines, access to container registries, etc.

Instructions

High-level summary of what you will perform in this lab, but more detailed than the objective.

1. Task or outcome.
    a. Something to do, using a graphical or web UI. Click **Save As** to save your work.

b. More things to do

2. Task or outcome.

a. Something to do, using a shell.

```
$ command --options fixed-arguments variable-arguments
output of the command
highlight something in the output
```

b. More things to do. Press `Ctrl` + `C` to interrupt.

Summary of what you acomplished.

What's next

Having evaluated models with EvalHub, the next chapter focuses on evaluating agentic applications using the MLFlow SDK.

Agent Evaluation with MLFlow

Goal

Evaluate agentic applications using the MLFlow SDK with deterministic scorers and LLM judges on Red Hat OpenShift AI.

Not started yet.

Agentic Evaluation Concepts

Objective

Describe the different kinds of assessment for agentic applications and explain how to code MLFlow evaluations using scorers, judges, and data sets.

Not started yet.

Heading

Lorem ipsum

Another heading

Lorem ipsum

What's Next

Next, you evaluate a chat application using a deterministic scorer and view the results as MLFlow traces.

Lab: Evaluate an Agent with Deterministic Scoring

Objective

Add tracing to a chat application, create a deterministic scorer, and run an MLFlow evaluation to verify response quality.

Not started yet.

Before you begin

Describe requirements, such as customer portal accounts, Red Hat Developer free sub, already configured machines, access to container registries, etc.

Instructions

High-level summary of what you will perform in this lab, but more detailed than the objective.

1. Task or outcome.
    a. Something to do, using a graphical or web UI. Click **Save As** to save your work.
    b. More things to do
2. Task or outcome.
    a. Something to do, using a shell.

```
$ command --options fixed-arguments variable-arguments
output of the command
highlight something in the output
```

    b. More things to do. Press `Ctrl` + `C` to interrupt.

Summary of what you acomplished.

What's next

Next, you evaluate the same application using built-in and custom LLM judges for more nuanced assessment.

Lab: Evaluate an Agent with an LLM Judge

Objective

Perform evaluations using built-in and custom LLM judges with self-hosted models, and inspect evaluation results through MLFlow traces.

Not started yet.

Before you begin

Describe requirements, such as customer portal accounts, Red Hat Developer free sub, already configured machines, access to container registries, etc.

Instructions

High-level summary of what you will perform in this lab, but more detailed than the objective.

1. Task or outcome.
    a. Something to do, using a graphical or web UI. Click **Save As** to save your work.
    b. More things to do
2. Task or outcome.
    a. Something to do, using a shell.

```
$ command --options fixed-arguments variable-arguments
output of the command
```

```
highlight something in the output
```

b. More things to do. Press `Ctrl` + `C` to interrupt.

Summary of what you acomplished.

What's next

Congratulations on completing this course! You are now able to deploy MLFlow and EvalHub on Red Hat OpenShift AI, evaluate both models and agentic applications, and visualize results using the RHOAI dashboard.